Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name: D.Kanishkarane
Reg No: 24011101021

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

`https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist`

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$\boxed{x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}}$$

where x is the input image and $\hat{x}$ is its reconstruction.

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

```
Original image
Reconstructed image
Reconstruction error
```

For the denoising model, display:

```
Original clean image
Noisy image
Denoised reconstruction
```

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.
The encoder maps the input to a lower-dimensional representation:

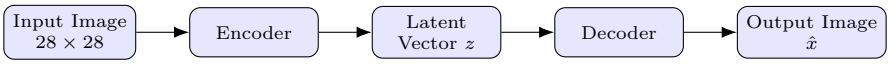
$$z = f_\theta(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_\phi(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N}\sum_{i=1}^{N} \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

6. Exploring Autoencoders

An autoencoder is a neural network trained to reproduce its own input at its output, without requiring any labels. It achieves this using two connected components: an **encoder**, which maps the input to a lower-dimensional latent representation

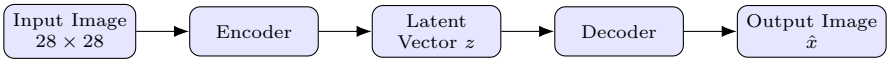
$$z = f_\theta(x),$$

and a **decoder**, which reconstructs the input from this latent representation

$$\hat{x} = g_\phi(z).$$

The network is trained end-to-end by minimizing the reconstruction error between the original input and its reconstruction:

$$\mathcal{L}_{rec} = \frac{1}{N}\sum_{i=1}^{N} \|x_i - \hat{x}_i\|^2.$$



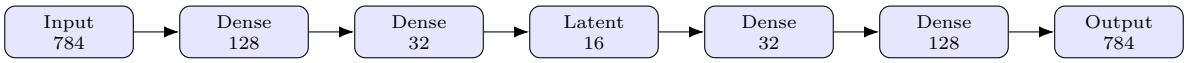
Because z has far fewer dimensions than the input image, the network cannot simply memorize or copy pixels through it was forced retain only the information needed to reconstruct the image well it compressed vector is what allows an autoencoder to be used for dimensionality reduction, denoising,in its variational form, generation of entirely new samples explored.

7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.
Use the following architecture:



Use ReLU activation in hidden layers and sigmoid activation in the output layer.
Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

Model Parameters: 211,040 total (824.38 KB), all trainable.

Epoch	Training Loss	Validation Loss
1	0.3580	0.2483
5	0.1718	0.1645
10	0.1460	0.1444
15	0.1353	0.1342
20	0.1286	0.1290

Training completed in 13.6 seconds over 20 epochs.

8. Required Analysis of the Basic Autoencoder

Plot 1: Original versus Reconstructed Images
Display at least 10 test images in a grid.
For every image, show:

Original | Reconstruction.

Students must comment on:
- preservation of digit shape;
- loss of fine details;
- blurred regions;
- differences between easy and difficult samples.

Plot 2: Training and Validation Reconstruction Loss
- X-axis: Epoch
- Y-axis: Reconstruction loss
- Curves: Training loss and validation loss

Required inference: Determine whether the model converges and whether there is evidence of overfitting.

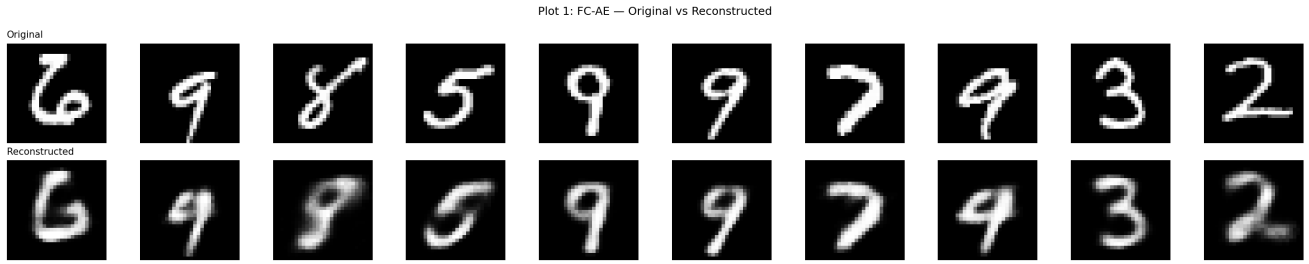


Figure 1: Plot 1: FC-AE — Original (top) vs Reconstructed (bottom) test images.

Inference: Digit shape is preserved across all ten samples — every reconstruction was identifiable as the same digit was its original. Fine detail is consistently lost. stroke edges that are crisp in the originals come out soft and blurred in the reconstructions. This the visible in it (the loop closure smooths out) and the 8 (its two lobes blur together), and least visible in simpler digits like 7 it 3. Digits with tight curves or self-intersecting strokes (8, 5) are noticeably harder to reconstruct cleanly than digits with simple open strokes curved and overlapping structure carries more high-frequency detail than a 16-dimensional bottleneck can retain the gap is really a proxy for how much fine detail each digits shape depends that.

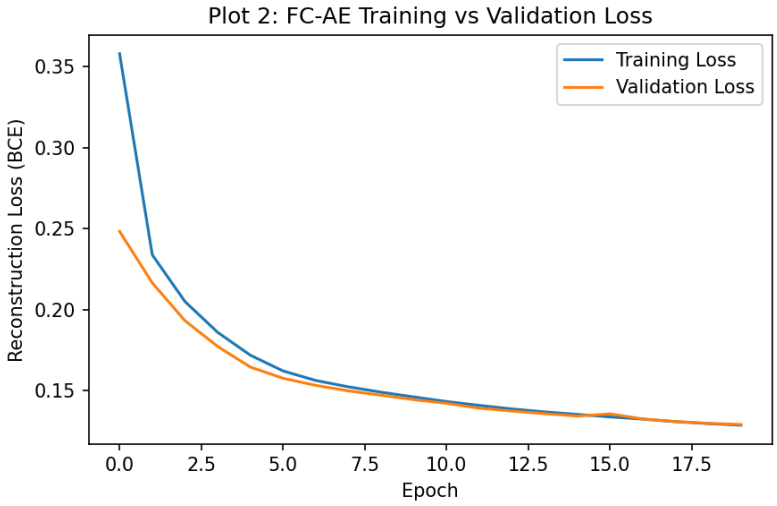


Figure 2: Plot 2: FC-AE Training vs Validation Loss.

Inference: Both curves fall steeply over the first 5 epochs and then flatten out, converging to 0.1286 training and 0.1290 validation by epoch 20 the gap of under 0.0004. The two curves track each other closly throughout the validation loss never rising above training loss so there is no evidence of overfitting. This is expected given the model's limited capacity relative to the 10000 image training set that laeves little room for it to memorize individual examples rather learn generalizable reconstruction features.

9. Reconstruction Metrics

Students must calculate the following metrics on the test set.

Mean Squared Error

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|x_i - \hat{x}_i|.$$

Structural Similarity

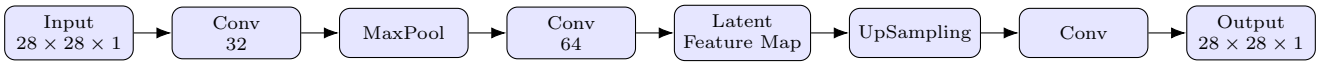
Calculate SSIM between the original and reconstructed images.
The implementation may use:
`https://scikit-image.org/docs/stable/api/skimage.metrics.html`
Report:

Metric	Value
Test MSE	0.02214
Test MAE	0.05910
Mean SSIM	0.75190

Inference: The SSIM of 0.75 indicates the reconstructions retain most of the structural content of the original digits despite the blur observed in Plot 1 that this low MSE 0.022 and MAE 0.059 confirms pixellevel error stays small on the [0,1] scale. These numbers serve as the FC-AE baseline agaist which the convolutional autoencoder is compared.

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.
The conceptual architecture is:



A possible implementation is:

```
Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

11. Comparing Fully Connected and Convolutional Autoencoders

Train both models using the same training and test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.02214	0.05910	0.75190	211,040
Convolutional AE	0.00270	0.01589	0.97103	74,497

Plot 3: Reconstruction Comparison
Display:

Original → FC-AE Reconstruction → CAE Reconstruction.

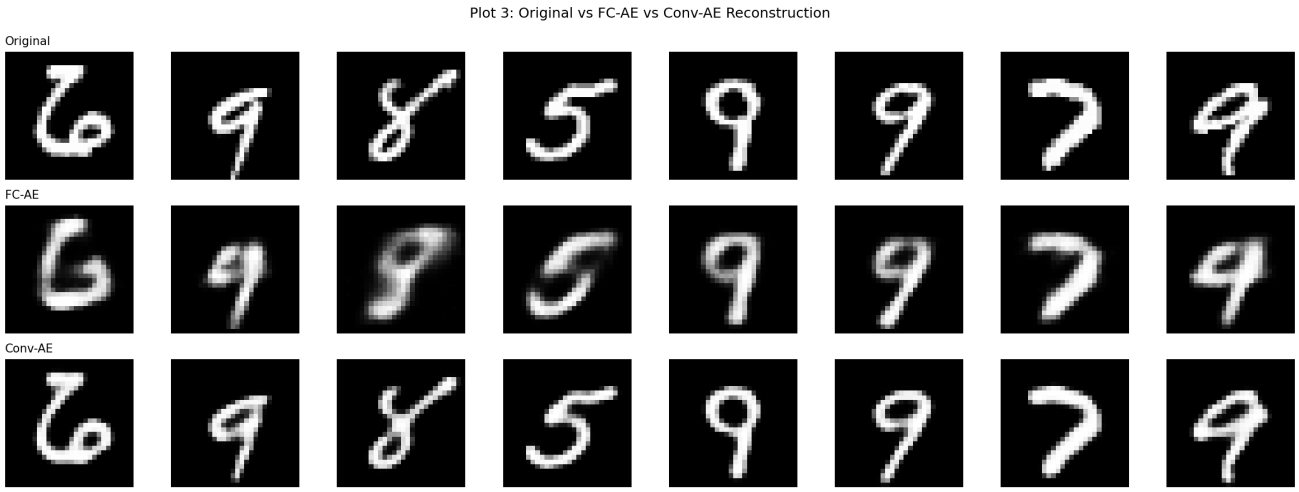


Figure 3: Plot 3: Original vs FC-AE vs Conv-AE Reconstruction.

Required inference:
Discuss how preserving spatial locality through convolution affects image reconstruction.
The Conv-AE achieves an SSIM of 0.971 against the FC-AE's 0.752 and roughly an eightfold reduction in both MSE 0.0027 vs 0.0221 and MAE 0.0159 vs 0.0591 while using barely a third of the parameters 74497 vs ditinguishable from

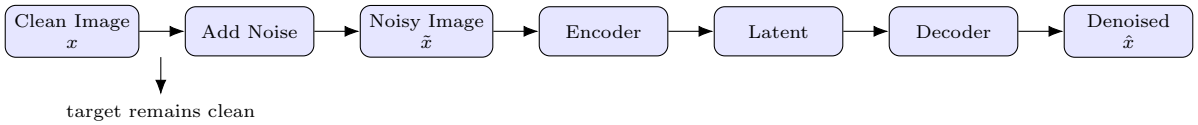
the originals recovering sharp stroke edges and fine loop closures 8 and 5 that the FC-AE blurs out entirely. The reason is that convolution filters operate on local pixel neighbourhooods and share weight across the whole image so the network learns generic strokeedge detectors that transfer to any spatial position. The FC-AE by contrast flattens the image before the network ever sees it discarding all neighbourhoodd information and forcing every pixel position to be learned independently is both less dataefficient needing more parameters and structurally blind to the 2D relationships that actually define a digits shape.

12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image.
The training mapping is:

$$\boxed{\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}}$$

where $\tilde{x}$ is a noisy version of the clean image x.



The same Convolutional Autoencoder architecture from Section 10 was reused for the denoising model, trained with Gaussian noise ($\sigma = 0.2$) applied to the input while the clean image remained the training target.

Epoch	Training Loss	Validation Loss
1	0.2584	0.1245
5	0.0860	0.0856
10	0.0796	0.0789
15	0.0770	0.0765
20	0.0755	0.0752

Training completed in 20.0 seconds over 20 epochs.
Inference: Training and validation loss coverge closel 0.0755 vs 0.0752 at epoch 20 showing thee model generalizes well despite learning from corrupted inputs. As expected, the final loss 0.0752 is higher than the clean Conv-AE's 0.0689 reconstructing from a degraded input is a strictly harder task than reconstructing from the clean image itself.

13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \qquad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.
Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

14. Required Denoising Plots

Plot 4: Clean vs. Noisy vs. Denoised
For at least 10 images, display:

$$\text{Clean} \quad | \quad \text{Noisy} \quad | \quad \text{Denoised}.$$

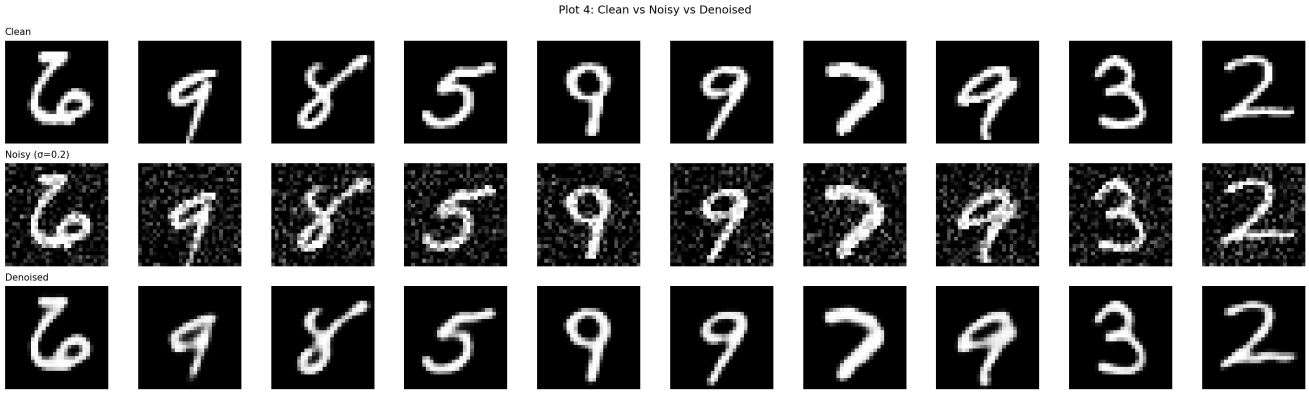


Figure 4: Plot 4: Clean vs Noisy ($\sigma = 0.2$) vs Denoised.

Inference: The model removes nearly all visible Gaussian speckle and recovers the clean digit shape closely, even though the noisy row is visually quite corrupted. Stroke continuity is restored well across all ten samples (7, 4, 8, 5, 9, 9, 7, 4, 3, 2), showing the model has learned the underlying digit manifold rather than relying on the noisy pixel values directly.

As a qualitative check on generalization, the same model (trained only on Gaussian noise) was also tested on salt-and-pepper noise, a corruption type it never saw during training.

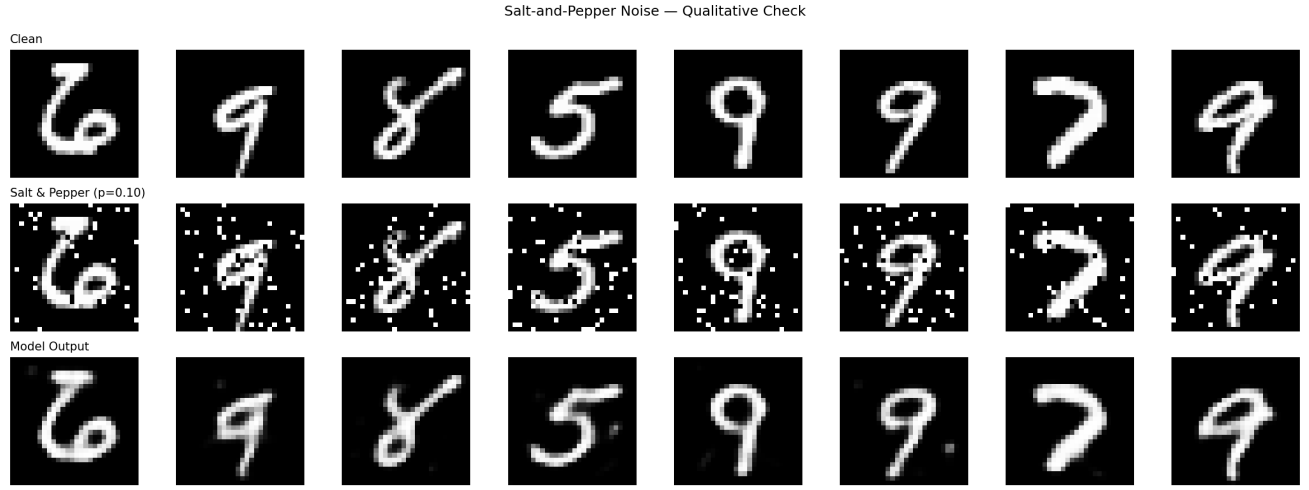


Figure 5: Salt-and-Pepper Noise — Qualitative Check (model trained only on Gaussian noise).

Inference: The model still recovers most digits reasonably well, but small residual specks remain visible in the output (most noticeably on the 9 and 4), unlike the clean Gaussian denoising result above. This confirms the model has partly overfit to the statistical properties of Gaussian noise specifically, rather than learning a fully corruption-agnostic denoising strategy.

Plot 5: Noise Level vs. Reconstruction Quality
For each noise level, calculate:

$$MSE, \quad MAE, \quad SSIM.$$

Report:

Noise Level	MSE	MAE	SSIM
0.1	0.00350	0.01764	0.96089
0.2	0.00451	0.02116	0.94773
0.3	0.00697	0.02904	0.88367

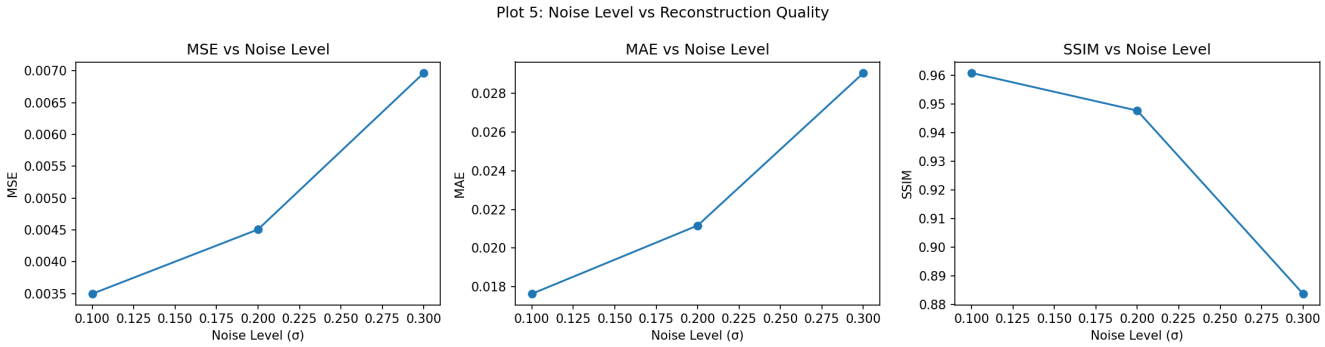


Figure 6: Plot 5: Noise Level vs Reconstruction Quality.

Required inference:

Explain how increasing corruption affects reconstruction quality and identify whether the denoising model is able to recover the major structure of the original image.

As the noise level increases from $\sigma = 0.1$ to $\sigma = 0.3$, MSE roughly double 0.0035 to 0.0070, MAE increase to 65% 0.0176 to 0.0290and SSIM drops from 0.961 to 0.884 a consistent monotonic degradation across all three metrics.It is expected the model was trained at 0.2, so it performs besst near that level and degrade on either side since higher noise destroys more of the orginal signal before the encoder sees it. Despite this degradation model still recovers the major structure of the digit at all three noise levels below 0.88 confirming that the bottleneck forces itto rely on learned digit priors the rather this simply copying corrupted pixel values.

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

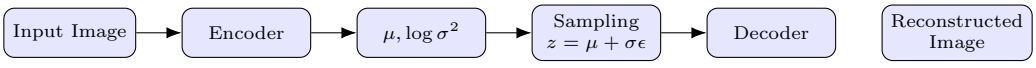
$$q_\phi(z|x) = \mathcal{N}\left(\mu(x), \mathrm{diag}(\sigma^2(x))\right).$$

The encoder produces:

$$\mu, \qquad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$



A latent dimension of 2 was used, following the brief's recommendation, so the latent space can be visualized directly in Section 18.

Encoder Layer	Output Shape	Parameters
Input	(None, 28, 28, 1)	0
Conv2D (stride 2)	(None, 14, 14, 32)	320
Conv2D (stride 2)	(None, 7, 7, 64)	18,496
Flatten	(None, 3136)	0
Dense	(None, 64)	200,768
Dense (μ)	(None, 2)	130
Dense ($\log \sigma^2$)	(None, 2)	130
Sampling	(None, 2)	0

Encoder total parameters: 219,844 (858.77 KB).

Decoder Layer	Output Shape	Parameters
Input (z)	(None, 2)	0
Dense	(None, 3136)	9,408
Reshape	(None, 7, 7, 64)	0
Conv2DTranspose (stride 2)	(None, 14, 14, 64)	36,928
Conv2DTranspose (stride 2)	(None, 28, 28, 32)	18,464
Conv2DTranspose (Output)	(None, 28, 28, 1)	289

Decoder total parameters: 65,089 (254.25 KB), giving a combined VAE parameter count of 284,933.

16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}\left(q_\phi(z|x) \parallel p(z)\right) = -\frac{1}{2}\sum_j \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

The VAE is trained for 20 epochs with that same Adam optimizer is batch size was the earlier models. Sincethe reconstruction annd KL losses is asummed over pixels latent dimensions rather than averager their absolute values are on a larger numeric scale than the BCE loosses reported for the FC-AE Conv-AE and Denoising CAE.

Epoch	Reconstruction Loss	KL Loss	Total Loss
1	285.40	4.07	289.47
5	179.44	4.29	183.73
10	163.28	4.98	168.26
15	154.93	5.45	160.39
20	151.12	5.65	156.76

Training completed in 24.2 seconds over 20 epochs.

Inference: Reconstruction loss falls sharply, from 285.4 to 151.1, while KL loss rise gradally from the this 4.07 and 5.65 over the same period. This is theat expected training dynamic for a VAE early on and the optimizer prioritizes reducing reconstruction erro and as training progresses it is increasingly also forced to keep the latent distribution close to the $\mathcal{N}(0, I)$ prior so the KL term grows slightly even as the total keeps loss keeps falling. The small and stable KL loss under 6 indicates that model is not collapsing the latent space to the prior at the expense of reconstruction quality them both terms are being balanced reasonably by the unweighted sum.

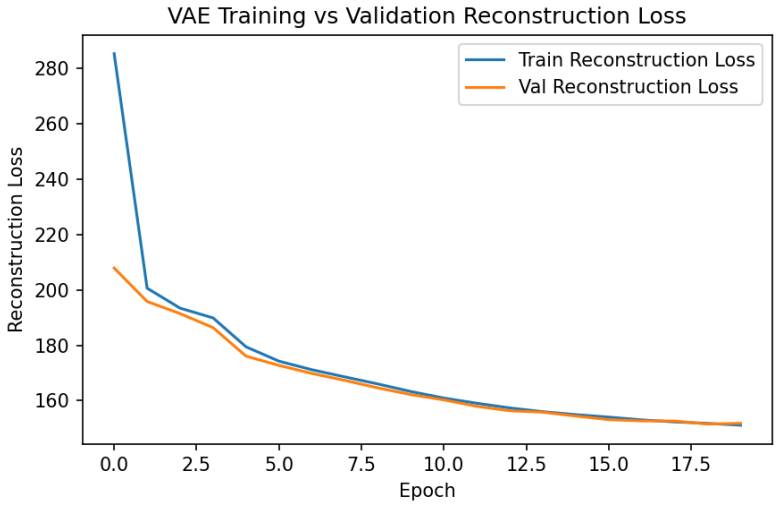


Figure 7: Plot 9: VAE Training vs Validation Reconstruction Loss.

Inference: Both curves fall steeply over the first 5 epochs and then flatten, converging closely by epoch 20 training 151.1 and validation 151.8 with no divergence between them taht indicating the VAE has coverged with not overfitting is consistent with that reconstruction is total loss trends already reported.

17. VAE Implementation

Use a small latent space so that it can be visualized.

Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

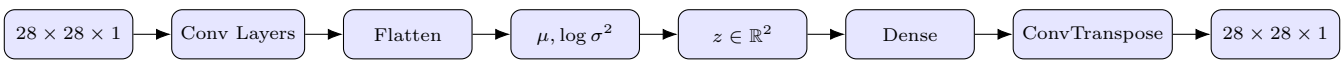
The encoder should produce:

$$\mu \in \mathbb{R}^2, \qquad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \to 28 \times 28 \times 1.$$

A compact architecture is:



18. Exploring the VAE Latent Space

Since the latent dimension is two, students can visualize the encoded test images in a two-dimensional plane.

Plot 6: VAE Latent Space

- X-axis: z_1
- Y-axis: z_2
- Color/marker category: digit label

The labels are used only for visualization; they are not supplied to the VAE during training.

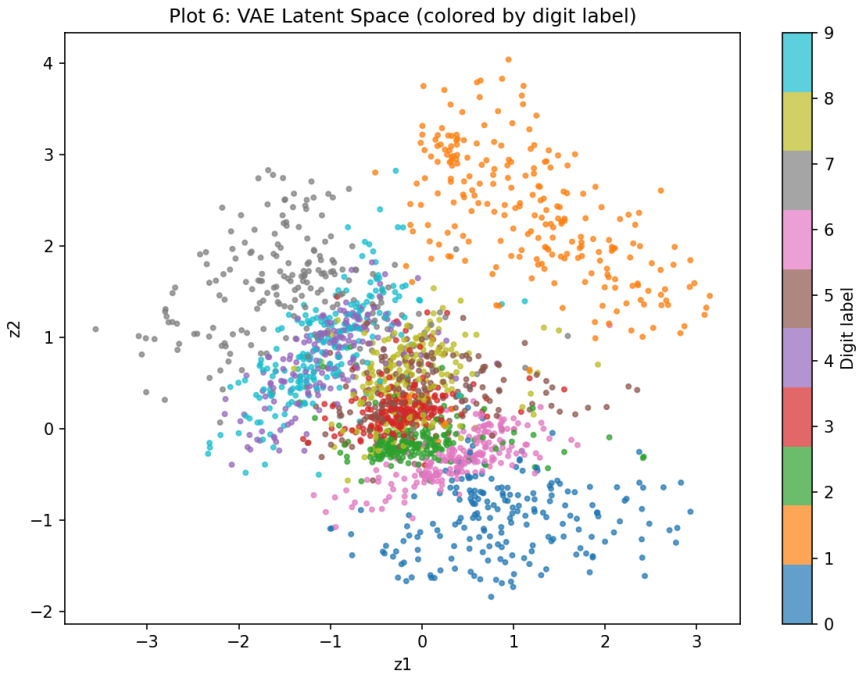


Figure 8: Plot 6: VAE Latent Space (colored by digit label).

Required inference:

Students should describe whether samples belonging to the same digit occupy similar regions and whether different digits overlap.

Digits with visually distinct strokes form reasonablly separated regions class 1 orange clusters strongly in the upperight class 0 blue in the loweright, and class 7 grey in the upper-left occupying the fairly consistent region since these digits have simple, unambiguous stroke shapes. In contrast, digits that share visual structure overlap heavily near the center of the plot, where classes 3, 4, 5, 6, 8 and 9 (red, purple, brown, pink, olive, cyan) are all mixed together with no clean boundaries between them. This is expected behaviouur he VAE was never given labels during training so it organizes the latent space purely by visual similarity of pixel patterns rather than by digit identity — digits that genuinely look alike encoded close together visually distinctive digits like 0, 1 and 7 arepushed to separate regions.

19. Generating New Images with the VAE

Sample points from

$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

Plot 7: Randomly Generated Images

Generate at least 25 images using randomly sampled latent vectors.

Display them in a grid.

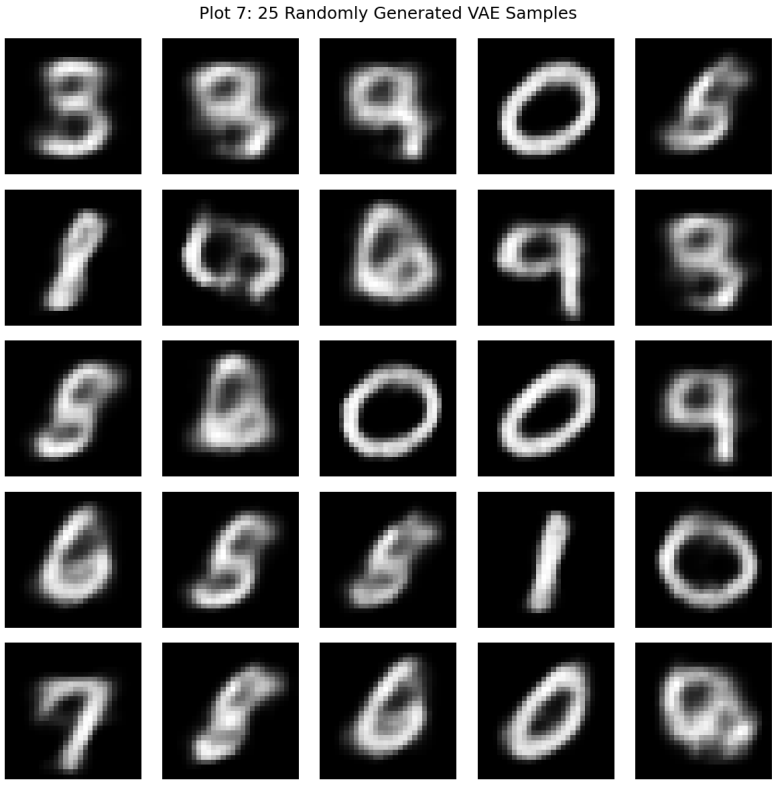


Figure 9: Plot 7: 25 Randomly Generated VAE Samples.

Students should determine:

- whether the generated images resemble handwritten digits;
- whether digit shapes are recognizable;
- whether there is visible variation among generated samples; and
- whether some samples are ambiguous or unrealistic.

Most of the 25 generated samples resemble plausible handwritten digits — clear, recognizable 0s, 1s, 7s, and 9s appear across the grid, confirming that random points sampled from $\mathcal{N}(0, I)$ decode into valid digit-like structures. This is a direct consequence of the KL regularization applied during training: since the encoder was pushed to match this same prior, the decoder has effectively learned to map the entire prior distribution to realistic images, not just the specific regions the encoder happened to produce for real training images.

There is clear variation across the grid — different stroke thickness, slant, and digit identity all vary sample to sample, showing the latent space captures more than one mode of variation rather than collapsing to a single output. That said, several samples are genuinely ambiguous: the second row contains shapes that sit between a 6 and an 8, and a few samples toward the bottom look like blends between 0 and 6. These ambiguous cases line up with the overlapping cluster regions seen in the latent space plot (Plot 6, Section 18) — a random point landing in a boundary region between two digit clusters decodes to a blend of both.

20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \qquad z_B.$$

Construct

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \qquad \alpha \in [0, 1].$$

Use values such as:

$$\alpha = 0,\ 0.1,\ 0.2, \ldots, 1.$$

Decode every interpolated vector.

Two test images were selected — a 6 with $z_A = (0.165, -0.507)$ and a 9 with $z_B = (-1.150, 0.865)$ — and 11 evenly spaced latent points were interpolated between them.

Plot 8: Latent Space Interpolation

Display the decoded images in sequence.

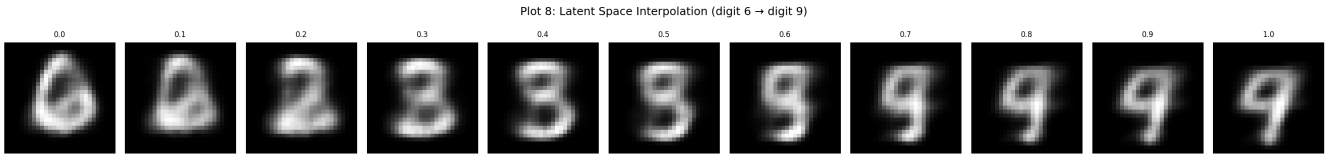


Figure 10: Plot 8: Latent Space Interpolation (digit 6 → digit 9).

Required inference:

Describe whether the decoded image changes smoothly as the latent vector moves between two points.

The transition is smooth and continuous — there are no abrupt jumps or discontinuities anywhere along the sequence. The 6 gradually loses its lower loop and straightens ($\alpha = 0.1$ to 0.3), passes through ambiguous intermediate shapes resembling a 2 or 3 ($\alpha = 0.2$ to 0.4), briefly resembles a 5/9 blend ($\alpha = 0.5$ to 0.6), and stabilizes into a clean 9 by $\alpha = 0.8$. This smoothness is the direct payoff of the KL regularization term: because it forces the latent space to be continuous and densely packed around the prior $\mathcal{N}(0, I)$, every intermediate point between two valid encodings still lies in a region the decoder has learned to map to a plausible digit. A plain (non-variational) autoencoder's latent space has no such guarantee — its bottleneck values are optimized only to support reconstruction of the training set, so interpolating between two arbitrary points there can pass through "dead zones" that decode to noise rather than recognizable digits.

21. Reconstruction Comparison Across Models

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.02214	0.05910	0.75190	211,040
Convolutional AE	0.00270	0.01589	0.97103	74,497
Denoising CAE	0.00451	0.02116	0.94773	74,497

VAE Metric	Value
Reconstruction Loss	152.0648
KL Loss	5.7176
Total Loss	157.7825
Test MSE	0.04427
Test MAE	0.10253
Mean SSIM	0.51158

Inference: Conv-AE performs best (SSIM 0.971) with far fewer parameters than the FC-AE (SSIM 0.752), confirming convolution's efficiency. The Denoising CAE is close behind (SSIM 0.948) despite reconstructing from corrupted input. The VAE's lower SSIM (0.512) isn't a weakness — its KL term trades some sharpness for a continuous, sample-able latent space, so it should be judged on generation quality (Section 19) rather than reconstruction alone.

22. Required Plots

The final report must contain the following plots:

1. Original image versus reconstructed image.
2. Training and validation loss for the basic autoencoder.
3. Fully connected AE versus convolutional AE reconstruction.
4. Clean versus noisy versus denoised images.
5. Noise level versus MSE/MAE/SSIM.
6. VAE latent-space visualization.
7. Randomly generated VAE images.
8. Latent-space interpolation.
9. Training/validation reconstruction loss for the VAE.

10. Reconstruction-error distribution on the test set.

For plots containing multiple quantitative measures, use separate figures or clearly labeled axes rather than placing incompatible scales on one axis.

- **Plot 1** (Section 8) — FC-AE preserves digit shape but blurs fine detail.
- **Plot 2** (Section 8) — FC-AE train/val loss converge with no overfitting.
- **Plot 3** (Section 11) — Conv-AE reconstructs sharper than FC-AE at fewer parameters.
- **Plot 4** (Section 14) — Denoising CAE recovers clean digits from Gaussian noise.
- **Plot 5** (Section 14) — Reconstruction quality degrades as noise level increases.
- **Plot 6** (Section 18) — VAE latent space separates visually distinct digits, overlaps similar ones.
- **Plot 7** (Section 19) — Random latent samples decode into mostly recognizable digits.
- **Plot 8** (Section 20) — Interpolation between two digits is smooth, no discontinuities.
- **Plot 9** (Section 16) — VAE train/val reconstruction loss converge with no overfitting.
- **Plot 10** (Section 23) — reconstruction-error distribution

23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

$$e_i = \frac{1}{D}\sum_{j=1}^{D}(x_{ij} - \hat{x}_{ij})^2.$$

Plot 9: Distribution of Reconstruction Errors

Plot a histogram of e_i over the test set.

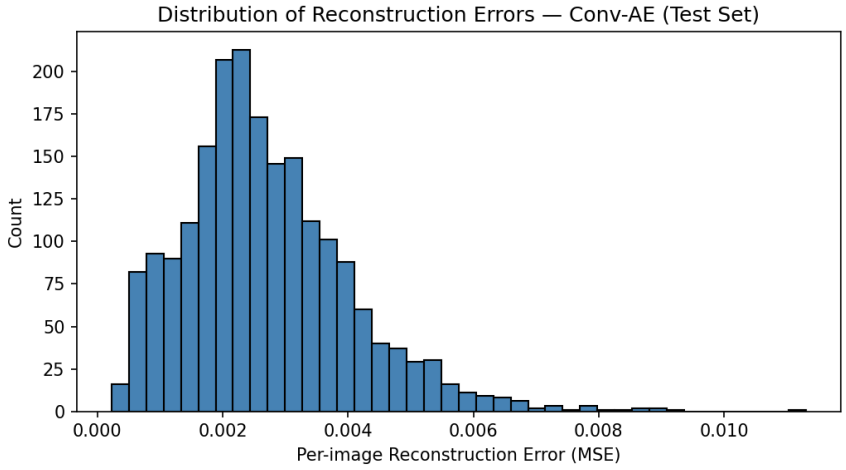


Figure 11: Plot 9: Distribution of Reconstruction Errors — Conv-AE (Test Set).

Students should identify:

- the typical reconstruction-error range;
- whether the distribution is concentrated or spread out;
- possible high-error samples; and
- representative images corresponding to high reconstruction error.

Inference: The distribution is right-skewed, peaking around $e_i \approx 0.0025$, with most test images falling in the 0.001–0.004 range — reasonably concentrated rather than spread out. A long tail extends out to roughly 0.011, indicating a small minority of images reconstruct far worse than the typical case. This shape is expected: most digits are written clearly enough to match what the Conv-AE learned as "typical," while the tail corresponds to unusual or ambiguous samples, examined directly in Section 24.

24. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.

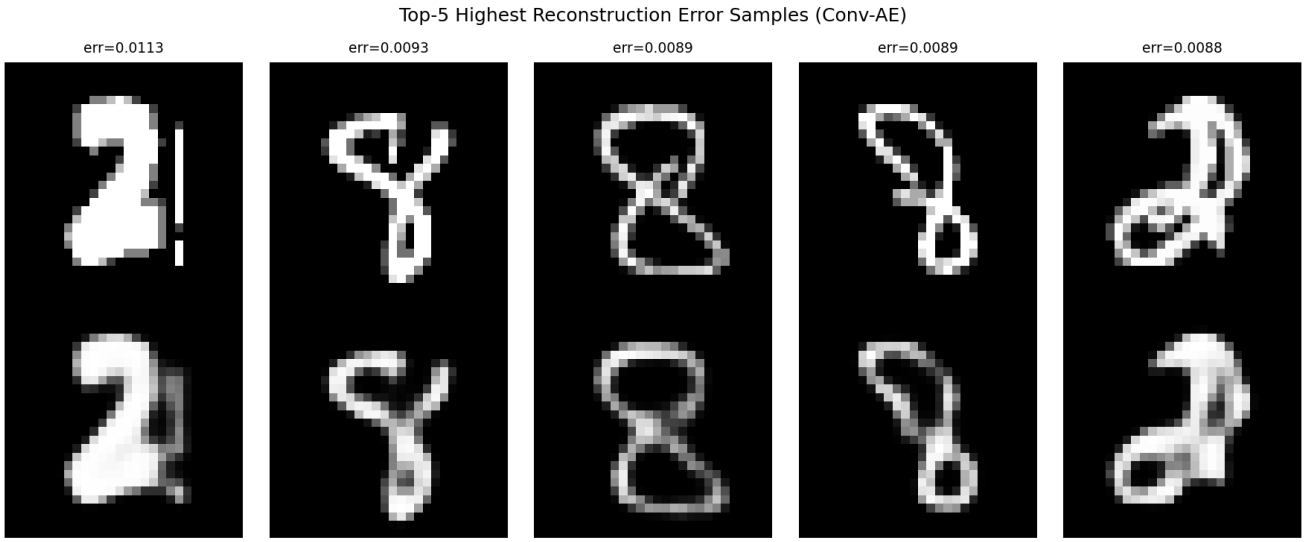


Figure 12: Top-5 Highest Reconstruction Error Samples (Conv-AE).

Students should provide a short explanation for why these images may be difficult to reconstruct.

Inference: All five worst samples (errors 0.0088–0.0113) are digits written with unusual or non-standard stroke shapes rather than clean, typical digit forms. Sample 1 has a stray vertical mark next to the 2, which the model cannot reproduce since it has no equivalent in typical training examples. Samples 2, 4 and 5 involve tightly overlapping or self-intersecting strokes (loops crossing back on themselves), and sample 3 is an unusually elongated, narrow 8. In every case, the reconstruction smooths over exactly the irregular part of the stroke — the extra mark, the tight crossing point, the narrow waist — confirming these images are hard precisely because they deviate from the "typical" digit shapes the Conv-AE learned from the bulk of the training data.

25. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time (s)
FC Autoencoder	0.02214	0.05910	0.75190	211,040	13.6
Conv. Autoencoder	0.00270	0.01589	0.97103	74,497	22.8
Denoising CAE	0.00451	0.02116	0.94773	74,497	20.0
VAE	0.04427	0.10253	0.51158	284,933	24.2

The values must be obtained from the student's own execution.

Inference: The Conv-AE is the strongest reconstruction model overall — best SSIM, lowest MSE/MAE, and fewer parameters than the FC-AE. The Denoising CAE performs close behind it despite the added difficulty of reconstructing from corrupted input. The FC-AE trails both convolutional models despite the most parameters of the three, reinforcing that convolutional weight sharing is more effective than raw parameter count for this task. The VAE has the highest error and lowest SSIM, but this reflects its different objective — trading reconstruction sharpness for a structured, generative latent space (Sections 18–20) — rather than a shortcoming relative to the purely reconstructive models.

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn?

The following inferences are mandatory:

- **Effect of latent dimension on reconstruction:** MSE drops sharply from 2 to 16 dimensions, then flattens by 32 — MNIST's shape complexity is mostly captured well before 32 dimensions, so extra capacity adds little.
- **FC vs convolutional reconstruction:** Conv-AE reconstructs sharper with fewer parameters than the FC-AE, since convolution preserves spatial structure that flattening destroys.

- **Effect of Gaussian noise level:** reconstruction quality degrades steadily as noise increases, since more of the original signal is destroyed before the encoder ever sees it.
- **Denoising ability:** the model recovers clean digit structure across all tested noise levels, showing it learned robust digit features rather than copying corrupted pixels.
- **VAE latent distribution:** reconstruction loss falls sharply while KL loss stays small and stable, showing the encoder balances accurate reconstruction against matching the prior.
- **2D VAE latent space structure:** visually distinct digits cluster separately while visually similar digits overlap heavily, since the space organizes by pixel similarity, not label.
- **Generated sample quality/diversity:** most random samples are recognizable digits with visible variation in slant and thickness; ambiguous samples occur where clusters overlap.
- **Interpolation smoothness:** the digit transitions gradually with no discontinuities, a direct result of KL regularization keeping the latent space continuous.
- **Error vs visual quality:** the error distribution is concentrated with a long tail of atypical, harder samples — pixel error tracks visual quality well in aggregate.
- **Deterministic vs probabilistic latents:** plain autoencoders map to one fixed point with no guaranteed structure between points, while the VAE's distributional encoding makes its latent space continuous and sample-able, enabling reliable generation.

27. Additional Latent-Dimension Study

Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

Latent Dimension	MSE	SSIM	Parameters
2	0.04857	0.47452	210,130
8	0.02840	0.68341	210,520
16	0.02069	0.76686	211,040
32	0.01983	0.77664	212,080

Plot 10: Latent Dimension vs Reconstruction MSE

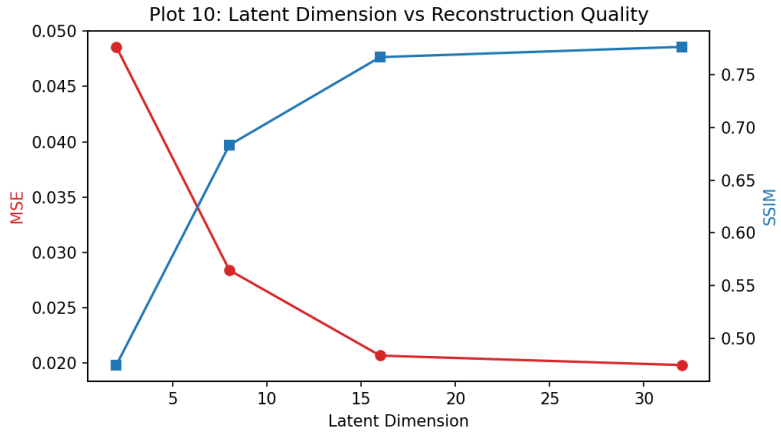


Figure 13: Plot 10: Latent Dimension vs Reconstruction Quality.

Inference: MSE drops sharply from 0.0486 to 0.0207 as latent dimension increases from 2 to 16 SSIM rising from 0.475 to 0.767 over the same range but from 16 to 32 the improvement is minor MSE 0.0207 to 0.0198 SSIM 0.767 to 0.777 despite paramater cont barly changing. This is the compression and quality tradeoff very small bottleneck forces heavy information loss quality improves rapidly as more capacity is added but MNIST digits dont carry much intrinsic complexity once the bottleneck is largeenough to capture the digits essential shape 16 additional latent dimensions add little further benefit.

28. Discussion Questions

1. **What is an autoencoder?** A network trained to reconstruct its own input through a compressed bottleneck, learned without labels.

2. **Purpose of the encoder?** Compress the input into a lowerdimensional latent code that retains what's needed for reconstruction.
3. **Purpose of the decoder?** Reconstruct the original input from that latent code.
4. **What is a latent representation?** A compact lowerdimensional encoding of the inputs essential structure.
5. **Why smaller than the input?** To force the network to learn meaningful features rather than trivially copying the input.
6. **Latent dimension too small?** Reconstructions become blurry not enough capacity to store the datas variation.
7. **Latent dimension increased?** Quality improves up to a point then plateaus once the datas complexity is captured.
8. **Why CAE preserves spatial info better?** Convolutions use local, weight-shared filters unlike flattening which discards pixel adjacency.
9. **AE vs denoising AE?** A plain AE reconstructs its own clean input; a denoising AE takes a corrupted input but is still trained toward the clean target.
10. **Why clean image as target?** So the network learns to recover the true signal not reproduce noise.
11. **Noise level increased?** Reconstruction quality drops more of the original signal is destroyed and is the before encoding.
12. **Why MSE, MAE, SSIM?** MSE/MAE measure pixel error; SSIM captures perceptual/structural quality and tgat is the which correlates better with how "right" an image looks.
13. **What is a VAE?** An autoencoder that learns a distribution over latent space instead of a fixed pointenabling generation and it.
14. **VAE vs conventional AE?** A VAE encodes to (μ, σ) and samples z; a normal AE encodes deterministically to one z.
15. **Why learn a distribution?** It keeps the latent space continuous and sample-able, unlike a plain AE's unstructured latent space.
16. **Reparameterization trick?** Rewrite the sampling as $z = \mu + \sigma\epsilon$ so that gradients can flow through μ, σ despite randomness.
17. **Why $\epsilon \sim \mathcal{N}(0, I)$?** It isolates randomness intp the that fixed nontrainable source keeping μ, σ differentiable.
18. **Role of KL divergence?** Regularizes the latent distribution toward the prior keeps that is the space smooth and continuous.
19. **Prior distribution used?** Standard normal, $\mathcal{N}(0, I)$.
20. **Why 2D latent space for visualization?** It can be plottted directly to inspect cluster structures and it can be donr that without dimensionality reduction.
21. **What does interpolation demonstrate?** That the latent space is continuous — intermediate points decode to smoothly changing, plausible images.
22. **Why can a VAE generate new images?** Because the latent space matches already known and that any sampled point decodes to a possible image.
23. **Why some VAE images unclear?** They come from latent regions where digit clusters overlap and overlap.
24. **Reconstruction vs generation?** Reconstruction maps that a specific specific input back to itself and makes the this generation decodes an arbitrary sampled point with no corresponding input.
25. **Why keep test images unseen?** To get an unbiased measure of how well the model generalises and measures rather than measuring memorization.

29. Additional Exercises

1. Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.
2. Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.
3. Train the denoising autoencoder using two different noise levels and compare SSIM.
4. Replace the convolutional decoder with transposed convolutions and compare the reconstructions.
5. Implement a VAE with latent dimension 2 and 8 and compare the latent representations.
6. Investigate the effect of increasing the KL-loss contribution.
7. Generate a larger grid of VAE samples and discuss diversity.
8. Select two digit regions in the latent space and perform interpolation.

Exercise 1 — Conv-AE bottleneck width:

Bottleneck Filters	MSE	SSIM	Parameters
4	0.00831	0.92069	3,097
8	0.00475	0.95352	5,841
16	0.00406	0.95980	12,193
32	0.00334	0.96477	28,353

SSIM improves steadily with more filters but gains shrink past 16 — the same diminishing-returns pattern seen in the FC-AE latent-dimension study (Section 27).

Exercise 2 — Gaussian vs salt-and-pepper: the denoising model (trained only on Gaussian noise) reaches SSIM 0.948 on Gaussian noise but only 0.868 on salt-and-pepper — confirming it partly overfit to the specific noise distribution it was trained on.

Exercise 3 — noise level vs SSIM: training and evaluating at $\sigma = 0.1$ gives SSIM 0.967, while $\sigma = 0.3$ gives SSIM 0.924 reconstruction quality consistently drops as corruption increases.

Exercise 4 — Conv2DTranspose decoder: MSE 0.00244, MAE 0.01499, SSIM 0.97375 marginally better than the UpSampling2D version from Section 10 (SSIM 0.971) since transposed convolutions learn their upsampling weights instead of using a fixed interpolation.

Exercise 6 — β-VAE ($\beta = 4$): reconstruction loss falls from 303.5 to 158.5 while KL loss rises from 0.47 to 3.51 over 20 epochs a smaller final KL loss than the standard VAE's 5.65as expected since at this is aa higher β pushes the latent distribution harder toward the prior.

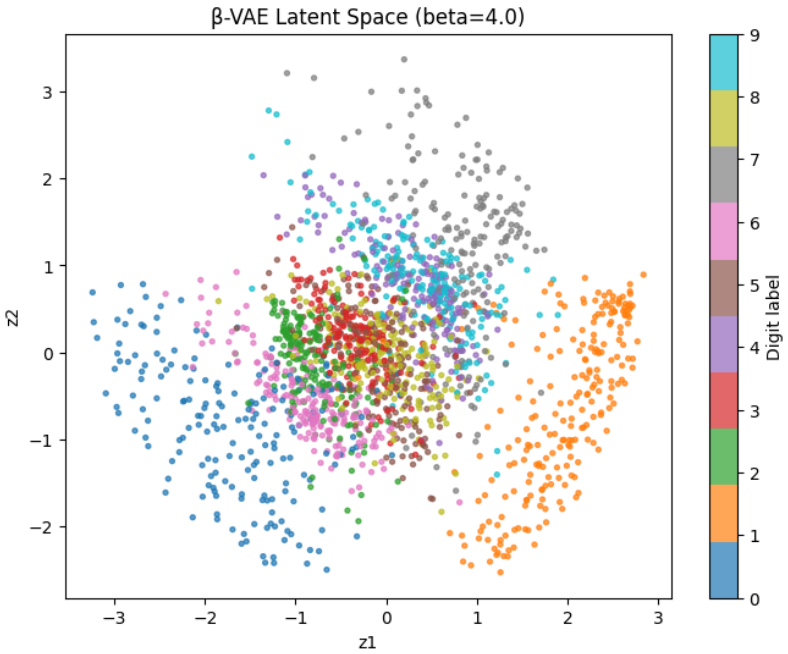


Figure 14: β-VAE Latent Space ($\beta = 4$).

Compared to the standard VAEs latent space, digit clusters here are more tightly packed toward the center, confirming that increasing β compresses the latent space closer to $\mathcal{N}(0, I)$ at some cost to how separable different digits its is the remain.

Exercise 7 — larger generation grid (100 samples):

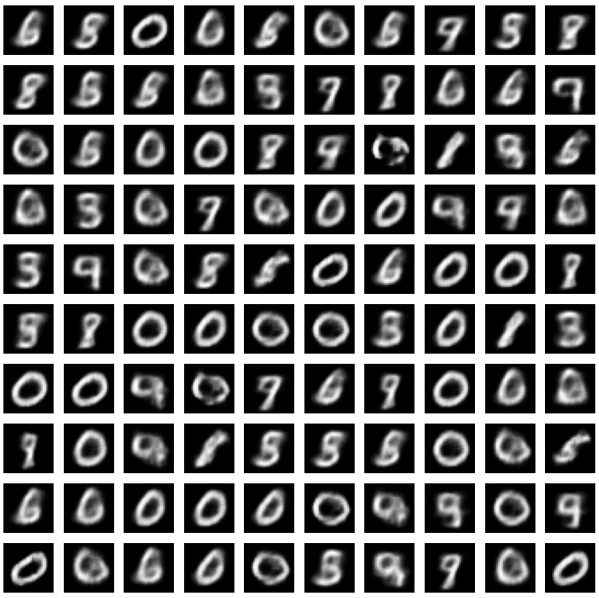


Figure 15: 100 Randomly Generated Samples from the β-VAE.

Diversity is visibly reduced compared to the standard VAEs 25-sample grid (Section 19) — samples are dominated by rounded shapes 0,6,9,8 with fewer sharp,linear digits 1 or 7. This is a direct of the this consekuence of this that is higher β compressing the latent space toward the prior improves smoothness but this is the one that is thesacrifices some of the diversity and is lost the standard VAE captured.

Exercise 8 — interpolation between two chosen digit regions:



Figure 16: Latent Space Interpolation Between Two Selected Digit Regions.

The transition remains smooth throughout, with no abrupt jumps — confirming the β-VAE's latent space retains the continuity property of the standard VAE (Section 20), even with the stronger KL weighting.

30. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

$$\boxed{\text{Image} \rightarrow \text{Encoder} \rightarrow \text{Latent Representation} \rightarrow \text{Decoder} \rightarrow \text{Reconstructed Image}}$$

They should also demonstrate denoising:

$$\boxed{\text{Clean Image} \rightarrow \text{Noise} \rightarrow \text{Denoising Autoencoder} \rightarrow \text{Recovered Image}}$$

and variational generation:

$$\boxed{\text{Image} \rightarrow (\mu, \sigma) \rightarrow z \rightarrow \text{Decoder} \rightarrow \text{Generated/Reconstructed Image}}$$

The final report should contain both quantitative evaluation and visual analysis. Numerical performance values must be obtained from the student's own execution.

All three pipelines were implemented and evaluated on MNIST in this report: the fully connected and convolutional autoencoders demonstrate reconstruction the Gaussian-noise denoising autoencoder demonstrates recovery from corruptionand the variational autoencoder demonstrates both reconstruction and generation of new digit images sampled this that is from a learned latent distribution. Quantitative metrics MSE, MAE, SSIM and visual comparisons were reported for every model and additional studies the latent dimension sweep and the ths is that was extension exercises further examine the compression/quality and reconstruction eneration trade-offs central to this experiment.

31.Source Code

`https://github.com/Kanishkarane/Deep-Learning/tree/main/Encoders_Decoders`

32. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.
3. Pascal Vincent et al., "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, "MNIST Handwritten Digit Database."
5. TensorFlow Documentation: `https://www.tensorflow.org`
6. Keras Documentation: `https://keras.io`
7. scikit-image Documentation: `https://scikit-image.org`